FIELD NOTES / 02

GPT-6 Astra vs Claude Fable 5

Beyond the benchmark.

What actually helped, what broke, and what I learned from a multi-agent DWH workflow.

THE CODE	Implementation, review and SQL evidence.
THE COORDINATION	Version drift, shared mistakes and human rework.
THE VERDICT	A personal field report, not a controlled model ranking.

By Derin Ege Evren

AI & Machine Learning Engineer

SEPTEMBER 2026 / ANONYMIZED PROJECT RETROSPECTIVE

01 / THE HONEST PREMISE

I did not need another confident answer.

I needed a change I could understand, transfer, test and trust.

There is a particular kind of frustration in AI-assisted development: the assistant says the fix is ready, you move the script into your work environment, and another revision arrives. The code may be improving. Your day is still disappearing into copy, paste, execute and report back.

That happened during my data warehouse work. It changed what I wanted to compare. Not which model could generate the most impressive SQL, but which working arrangement helped me reach a verified result with less unnecessary work.

My background is AI and machine learning engineering, and I approach warehouse development as both a data problem and a systems problem. This article follows my experience using the setups I identify as GPT-6 Astra in Codex and Claude Fable 5. It is a companion to my first article on effective AI use in DWH, not a laboratory benchmark.

AN IMPORTANT ATTRIBUTION LIMIT

The historical records identify the agents as Codex and Claude, not the model behind every turn. Astra and Fable 5 are my reported model labels. This retrospective cannot independently verify that each historical turn used those exact versions. Here, agent-level observations are not universal model-level conclusions.

The records cover selected development and review episodes, mainly from late August and early September 2026. Both agents saw previous decisions and each other's work. Their assignments changed. That makes this a collaboration case study, not two isolated contestants solving an identical fresh task.

Official sources identify GPT-6 Astra and Claude Fable 5 as model names [1, 2]. They do not verify the identity, settings or performance of my historical sessions. I make no claim here about Fable 5.1 or any other replacement model.

02 / THE ACTUAL SETUP

Two agents. One human-owned workflow.

The orchestration was mostly files, clear responsibilities and a shared channel.

The agents exchanged requests and reviews through a shared local text channel. Task notes preserved the current state, decisions and test evidence. A script fingerprint, or hash, identified an exact file version. None of that made the system an autonomous database operator.

IMPLEMENTER

Produces the scoped change and names the exact revision.

REVIEWER

Challenges assumptions and checks that same revision.

HUMAN OWNER

Controls scope, transfers scripts and runs corporate tests.

Figure 1. Responsibilities, not permanent model personalities. Implementation ownership changed during the project.

At an earlier stage, Claude owned implementation. After repeated delivery loops, I moved that responsibility to Codex and kept Claude involved in review and records. Later successes cannot be interpreted as a clean comparison: Codex inherited context, previous work and already-discovered problems. [E1]

What the framework actually added

It gave disagreements somewhere to live outside the chat. A reviewer could point to an exact version, a decision could state its scope, and a test could record what it did not cover. The most useful message was often not another solution. It was a precise objection to a claim that was too broad.

WHAT IT DID NOT ADD

A second agent was not an independent authority merely because it had a different name. Shared context could spread the same assumption. Shared files could change during review. A human still had to decide whether the evidence justified the next step.

For a small, low-risk edit, this much coordination can be excessive. In this project, its value was clearest around semantic changes, difficult SQL behavior and handoffs that otherwise would have relied on memory.

03 / CASE STUDY

Fast revisions became slow delivery.

The first performance problem was not model latency. It was the work arriving at my desk.

The project log records repeated Claude-led fix-and-test cycles, including at least six revisions or part refreshes around one test package. Each update required manual transfer and another attempt in the work environment. That delivery burden was the recorded reason for handing implementation to Codex. [E1]

A local optimization	The end-to-end consequence
Send every small fix immediately.	The human must identify, transfer and rerun another revision.
Keep improving the working file.	The reviewer may no longer be reading the requested version.
Announce that the code is fixed.	The recipient still needs to know what changed and what to test.

My preference after that experience was to keep one implementation owner and ask for a consolidated delivery: one current script, a short change summary, known limits and a runnable test. A reviewer should challenge that package before it reaches me whenever practical.

This is a real criticism of the Claude-led delivery in this episode. It is not proof that Fable 5 is inherently slower, or that Codex always produces fewer revisions. We did not record comparable response latency, token usage or human effort across controlled runs.

THE METRIC I WOULD MEASURE NEXT

Time to an accepted change, including human transfer, execution, interpretation and rework. A model can be quick to answer while the workflow is expensive to operate.

For an early-career engineer, this matters immediately. Ask for fewer, more complete handoffs. Do not become the mechanical courier between two agents that keep revising each other.

04 / CASE STUDY

Both agents agreed. Both were wrong.

Agreement is not the same thing as independent verification.

In a review of a transactional truncate-and-reload pattern, both agents had accepted an incorrect claim about what a concurrent NOLOCK reader could observe. A later Codex correction revisited the SQL Server locking semantics and cited Microsoft documentation. Claude explicitly withdrew the earlier claim. [E2]

The specific technical correction

TRUNCATE TABLE acquires a schema-modification lock, Sch-M. NOLOCK queries still acquire schema-stability locks, Sch-S, during compilation and execution. The latter are blocked by a concurrent Sch-M lock. For the transactional pattern under review, the relevant concern was blocking while that lock was held, not NOLOCK bypassing it to see the proposed intermediate state. [3, 4]

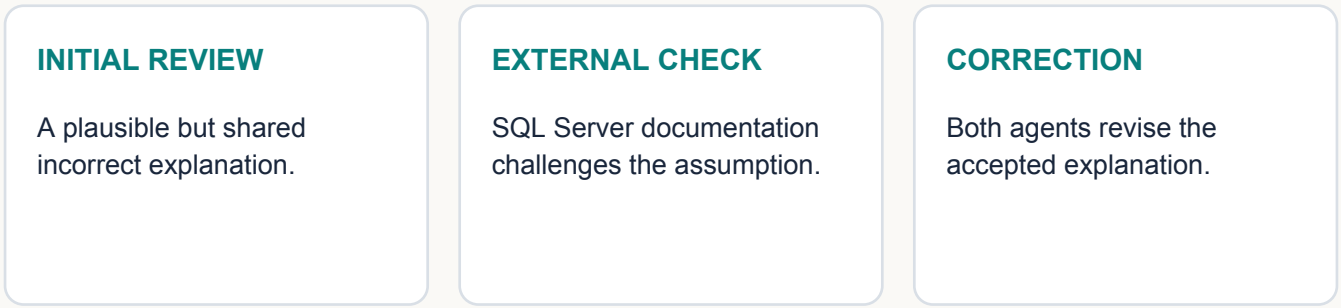


Figure 2. The valuable step was checking an external source, not obtaining two matching answers.

Codex deserves credit for the correction. It also shares responsibility for the earlier agreement. Claude deserves criticism for the initial mistake and credit for withdrawing it clearly. Presenting only one half of that story would make a misleading comparison.

This did not certify the entire load as safe. Source reads, transaction boundaries, timeouts and the wider deployment still required their own assessment. Correcting one mechanism does not prove the rest of the system.

A BETTER REVIEWER PROMPT

State the assumption you are challenging, identify an authoritative source or a reproducible test, and explain exactly which earlier conclusion changes. Do not approve the whole design because one objection is resolved.

05 / CASE STUDY

The approval pointed to the wrong version.

A good analysis is unusable if its approval points to an old revision.

Two recorded review cycles ended with a stale Claude acknowledgement: a newer request existed, but the approval still referred to an older script version. In the second cycle, the retrospective initially described a column-count mismatch as a successful catch. Rechecking the sequence showed that different versions had been mixed. Claude withdrew that interpretation. [E3, E6]

Sequence	What happened
1. Review requested	Revision A is named for review.
2. Request superseded	Revision B replaces A before acknowledgement.
3. Stale approval	The reviewer acknowledges A or mixes A with B's content.
4. Reconciliation	The mismatch is challenged; approval is tied to B.

Figure 3. Simplified version sequence. A and B replace internal identifiers; this is not a latency timeline.

The reviewer had not reliably bound the content it examined to the version it approved. Files also changed during active review. Claude reported that its watcher missed superseding messages; that explanation is a self-report, while the stale acknowledgements are visible in the channel.

A fingerprint is useful only if it belongs to the bytes actually reviewed. Recording a hash and then reading a changing file can still create a false sense of precision. For future work, I would review a frozen snapshot and bind the response to that snapshot.

```
REVIEW REQUEST: revision B / immutable snapshot
SCOPE: join keys only; filters unchanged
RESPONSE: reviewed B / findings / test boundary
If B changes: invalidate approval; request review again
```

There were two documented stale-approval cycles. We did not measure their wall-clock cost. The lesson is a version-control protocol, not a numerical score for either model.

06 / CASE STUDY

A stricter guard was not automatically safer.

Validation code can change the business contract just as ordinary SQL can.

Another episode involved ambiguous source mappings. Two conflict checks stopped a full-range test, while the recorded debug result showed the other ten structural checks passing. Claude predicted the two problematic counters before the follow-up diagnostic result confirmed them. [E4]

The discussion separated two issues: a lookup could have conflicting candidate values without multiplying result rows, and the legacy behavior did not necessarily treat that ambiguity as a fatal error. A deterministic selection rule and a business decision about unacceptable ambiguity are not the same thing.

After the agreed decision was revised, Codex delivered a narrow change to make those two conditions diagnostic warnings while retaining the other guards. Claude reviewed the limited patch. This is a useful example of diagnosis, scoped implementation and review contributing different pieces. [E4]

DO NOT COPY THE POLICY BLINDLY

This case is not advice to downgrade failed checks. Whether ambiguity should stop a load depends on the approved business contract. The useful behavior was identifying the exact disputed conditions and obtaining a scoped decision instead of removing validation broadly.

The original fail-fast change came from a separate session whose authorship and model identity were not established by the retained evidence. It belongs in the workflow failure column, not automatically in either model's failure column. The observed stoppage was a test run, not a demonstrated production outage.

What I learned to ask

For every new guard: what invariant does it protect, which decision authorizes it, and what evidence distinguishes invalid data from an allowed legacy ambiguity? More defensive-looking code is not necessarily a more faithful implementation.

07 / PERFORMANCE

The sevenfold speedup that was not a model win.

Same instrumented SQL, a repeat execution, and a tempting but unsupported headline.

One diagnostic run took 262,933 ms. A later repeat of the same instrumented batch took 37,617 ms; both reported 30 final rows. The contrast was dramatic. It was also a database execution observation, not a measure of how quickly either model reasoned or wrote code. [E5]

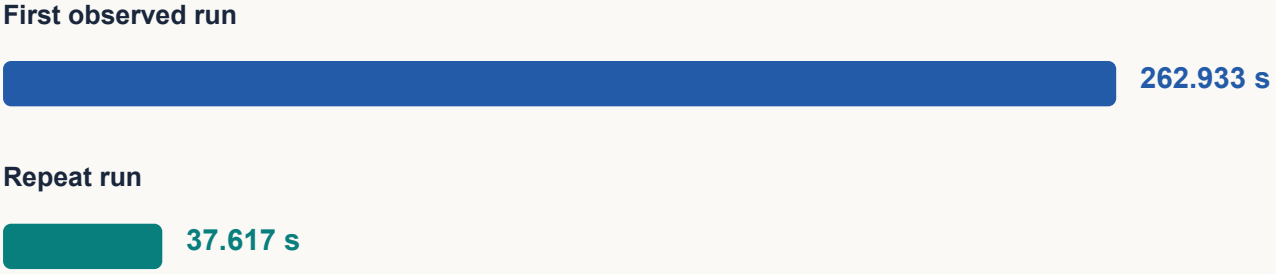


Figure 4. Observed SQL elapsed time. Roughly 7x difference; two executions, not a controlled model benchmark or an isolated causal test.

A warmed cache was a plausible explanation, supported by large reductions in repeated source-reading stages. But cache state, compilation, plan reuse and concurrent server conditions were not fully isolated. The older approximately 40-second baseline was not preserved as a verified equivalent artifact, so it could not serve as a strict A/B control.

There was another overstatement to correct. Small measured gaps between timed stages did not establish the total cost of instrumentation inside those stages. Claude's early explanation and Codex's monitoring summary expressed more causal certainty than the evidence supported. The retrospective narrowed those claims. [E5, E6]

THE DEFENSIBLE CONCLUSION

The same instrumented batch was capable of a much shorter repeat execution. We did not isolate why the first run was slower, measure all instrumentation overhead, or establish a model-driven speed advantage. Matching row counts alone also did not prove equal output values.

08 / REVIEWING THE REVIEW

The retrospective needed its own fact-check.

An agent can produce a convincing story about its work and still misattribute the evidence.

For this article, I asked Claude to identify its contributions, mistakes, observed Codex strengths and weaknesses, and the evidence behind each claim. I then had Codex challenge vague claims against the shared records. Two clarification rounds materially changed the account. [E6]

Initial retrospective claim	What survived clarification
Current model label proves historical identity.	Historical per-turn identity was not independently verified.
A column mismatch was a reviewer success.	Different script versions had been mixed; the success claim was withdrawn.
A later acceptance validated an earlier static review.	The acceptance belonged to another artifact; no shared proof chain.
A guard caused a production stoppage.	The recorded event stopped a full-range test; actor attribution remained unknown.
A few extra turns quantify the cost.	Two stale-approval cycles were visible; elapsed cost was not measured.

Claude acknowledged and corrected these claims. That made the contribution more useful, not less. Still, the corrections are evidence about this exchange, not proof that one model is uniquely honest or another uniquely reliable.

Codex was also a participant in the original work and is helping draft this account. It is not a neutral external evaluator. I keep the distinction between a logged event, an agent's own explanation and my personal takeaway visible throughout the article.

THE PUBLISHING LESSON

Ask an agent for evidence that could disprove its own account. Keep later corrections attached to the original claim. A polished narrative should never outrank the underlying record.

09 / THE PRACTICAL COMPARISON

Where each setup helped me.

A qualitative breakdown of this project, not a leaderboard of general capabilities.

Dimension	Codex / reported Astra	Claude / reported Fable 5
Implementation	Took ownership after handoff; delivered the scoped guard-policy patch.	Earlier implementation produced repeated delivery and retest loops.
Technical reasoning	Corrected the shared locking explanation using documentation.	Predicted the two disputed conflict counters; reviewed the limited fix.
Review discipline	Challenged stale approvals and later retrospective attributions.	Reported detailed static checks, but issued two stale approvals.
Weaknesses visible here	Shared an initial technical error; overstated what repeat timing proved.	Shared that error; mixed versions and evidence in its retrospective.
Speed and cost	No comparable model latency, token or spend measurements.	No comparable model latency, token or spend measurements.

My practical preference in this workflow became Codex for implementation ownership and Claude for focused second review. That is a role preference drawn from these episodes, not a claim that the roles should be permanent or that the same result will hold for another team.

The benefits were specific: complementary diagnosis, challenges to assumptions, durable explanations and tests with clearer boundaries. The costs were also specific: duplicate reading, stale state, review coordination and a human caught between revisions. An extra agent earns its place only when those benefits exceed its coordination burden.

Evidence: E1-E6. Assignments, accumulated context, tools and task phase were not controlled; causal model attribution is limited.

10 / A BETTER NEXT EXPERIMENT

Measure the workflow you actually operate.

If I repeat this comparison, I want an accepted-change record, not a stopwatch screenshot.

I would give each setup the same frozen, synthetic task package: schema, mapping, constraints, representative data and explicit acceptance tests. I would record exact model versions, settings and tool access. Rotating run order and repeating tasks would help expose one-off advantages.

Record	Why it matters
Accepted correctness	Check values, multiplicity, edge cases and scope preservation, not just execution.
Human effort and handoffs	Count transfer/retest actions and record active human minutes.
Model and tool measurements	Capture latency, retries, tokens and actual billed cost when available.
Database measurements	Keep SQL elapsed time, CPU, reads and plans separate from model latency.
Revision-bound evidence	Associate every approval and test with the exact executed artifact.

Protect the context before comparing the models

The agents were not directly executing against the corporate database in these episodes; I controlled execution and returned results. That limits execution authority, but it does not by itself prove data confidentiality. SQL text, screenshots, object names and error messages can still reveal sensitive information.

The examples here omit internal servers, customer records, employee messages and full proprietary scripts. For a future benchmark, I would use synthetic values and generalized names, and verify organizational approval, provider settings and permitted data categories before sharing context. A local coordination file does not establish that model processing is local.

FOR NEWER ENGINEERS

AI can help you attempt harder work with feedback. It does not replace the experienced engineer who recognizes an ambiguous business rule or an unsafe test. Seniority, age and agent usage are different variables; none is a fair proxy for competence.

11 / MY TAKEAWAY

I would keep the partnership. I would tighten the contract.

The outcome I care about is less uncertainty and less avoidable rework.

I started this comparison wanting to explain how Astra and Fable 5 performed in my work. The most useful answer turned out to be less dramatic than a winner: one agent was valuable as an implementation owner, another as a challenger, and both needed boundaries that their confidence could not supply.

The work improved when I could ask four concrete questions. Which version is current? Who owns the edit? What evidence supports the claim? What do I still need to decide or execute? The arrangement became fragile whenever those answers lived only in conversation.

ONE OWNER

Keep editing responsibility explicit for each scoped change.

FROZEN REVIEW

Review a named snapshot; a new edit needs a new approval.

HUMAN PROOF

Run authorized tests and keep untested boundaries visible.

Figure 5. My next iteration of the workflow: a proposed operating discipline, not a claim that every historical step followed it.

For someone new to DWH, the opportunity is not to pretend to have years of experience. It is to get more deliberate practice: propose a mapping, ask what can go wrong, run an appropriate test and explain the result yourself. The agent should make your reasoning more inspectable, not become a substitute for it.

For an experienced developer, the question is equally practical: does the assistant remove work you would otherwise do, or add another system you have to supervise? Declining a second agent can be a good engineering decision. Using two can be one too, when the task justifies independent challenge.

THE PRINCIPLE I AM TAKING FORWARD

Choose the arrangement that reaches a verified result with the least unnecessary human rework. Keep the models accountable to the evidence, and keep yourself accountable for the decision.

SOURCES / METHOD NOTE

What this account rests on.

Public technical references plus private project records, with their limits preserved.

[1] OpenAI - GPT-6 Astra
Model identity reference; not evidence of historical session configuration.

[2] Anthropic - Claude Fable 5 and Claude Mythos 5
Exact Fable 5 release reference; not a substitution with Fable 5.1.

[3] Microsoft - TRUNCATE TABLE
Schema-modification locking and transactional behavior.

[4] Microsoft - Table hints
NOLOCK does not remove schema-stability locking.

Private evidence register

Label	Retained record and scope
E1	27 August delivery/handoff log: repeated revisions and human transfer burden.
E2	3 September correction and acknowledgement: shared locking error.
E3	3 September review sequence: two stale acknowledgements and corrections.
E4	3 September diagnostic exchange and scoped policy revision: two conflict checks.
E5	3 September timing discussion: 262,933 ms and 37,617 ms executions.
E6	8 September retrospective plus two clarification rounds: withdrawn or narrowed claims.

These project records are retained privately, not published as independently accessible datasets. Case selection is retrospective and not exhaustive. Private timestamps are unsuitable for latency measurement. Model labels are author-reported; reasoning settings and per-turn identities were not consistently recorded.

Written with AI assistance and an evidence-focused review of agent accounts. Examples are anonymized. This is a personal field report, not an employer statement, security certification or controlled benchmark. Public sources checked 8 September 2026.